Maheen Saqib

23-NTU-CS-1260

Homework-01 – After mid

Embedded IOT Systems

Fall2025 AI 5th

Question-1 ESP32 Webserver (webserver.cpp)

Part A: Short Questions

1. **What is the purpose of WebServer server(80); and what does port 80 represent?**

WebServer server(80); creates a web server object on the ESP32 that listens for incoming HTTP requests.
Port **80** is the default port used for **HTTP (HyperText Transfer Protocol)**. When a user types an IP address in a browser without specifying a port number, the browser automatically uses port 80.

2. **Explain the role of server.on("/", handleRoot); in this program.**

This statement registers a handler function for the root URL (/). Whenever a client accesses the ESP32's IP address, the function handleRoot() is executed to generate and send the web page response.

3. **Why is server.handleClient(); placed inside the loop() function? What will happen if it is removed?**

server.handleClient(); continuously checks for incoming client requests and processes them. It must be inside loop() so the ESP32 can serve requests repeatedly. If this line is removed, the web server will not respond to any client requests, resulting in connection timeouts.

4. **In handleRoot(), explain the statement: server.send(200, "text/html", html);**

This command sends an HTTP response to the client with:

- **200** → HTTP status code meaning *successful request*
- **"text/html"** → content type indicating HTML data
- **html** → the actual web page content sent to the browser

5. What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside handleRoot()?

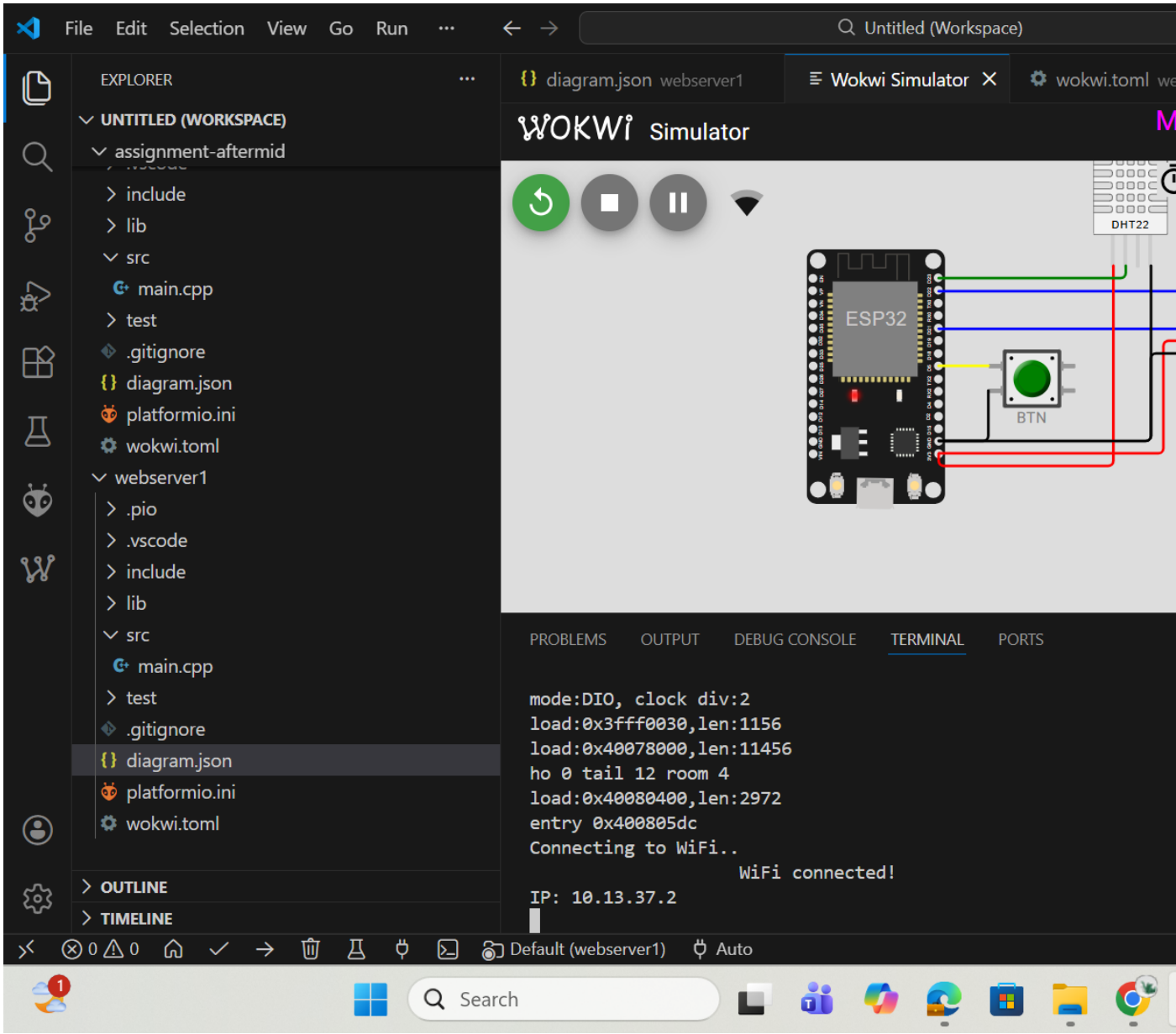
• **Last measured values**: Uses stored values (`lastTemp`, `lastHum`), providing fast page loading and consistency with OLED data.

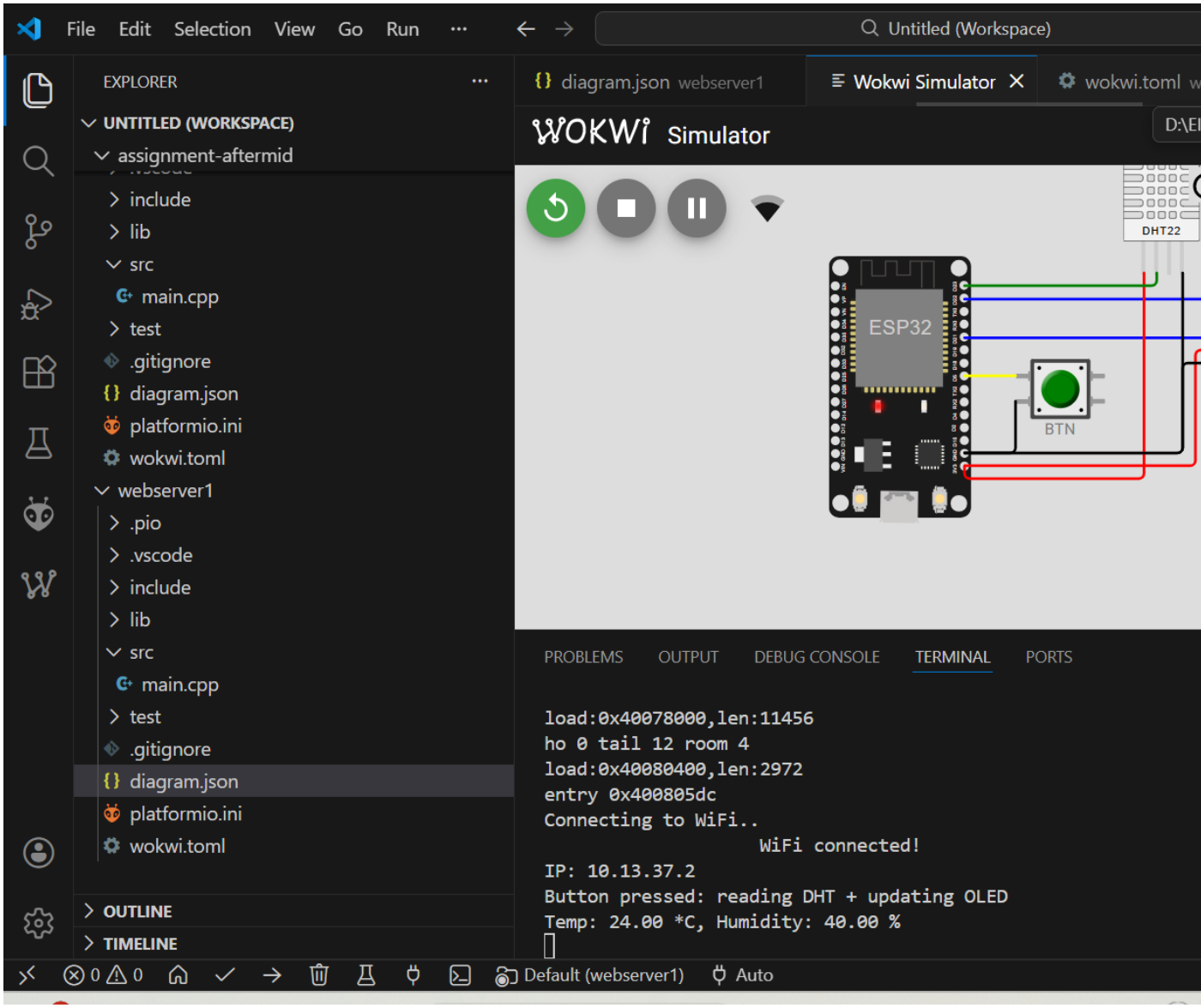
• **Fresh sensor reading**: Reads the DHT sensor every time the page loads, giving real-time data but causing delays and increasing the risk of sensor read failures due to DHT timing limitations.

Part B: Long Question

ESP32 Webserver-based Temperature/Humidity Monitoring System:

1. **Wi-Fi Connection & IP Assignment:**
    - ESP32 connects to "Wokwi-GUEST" (simulated) using WiFi.begin()
    - DHCP assigns an IP address automatically from router
    - IP displayed on OLED and Serial Monitor for client access
2. **Web Server Initialization:**
    - Server object created on port 80
    - Root handler registered with server.on("/", handleRoot)
    - Server started with server.begin()
3. **Button-based Sensor Reading:**
    - Button on GPIO5 (active LOW with INPUT_PULLUP)
    - Falling edge detection triggers sensor read
    - Reads DHT22 values, updates lastTemp/lastHum globals
    - Updates OLED display immediately via showOnOLED()
4. **Dynamic HTML Generation:**
    - handleRoot() builds HTML string with current sensor values
    - Uses string concatenation to insert temperature/humidity values
    - Includes error handling for invalid data
5. **Meta Refresh Purpose:**
    - <meta http-equiv='refresh' content='5'> auto-refreshes page every 5 seconds
    - Allows webpage to show updated sensor values without manual refresh
    - Note: In current code, values only update when button pressed, not on refresh
6. **Common Issues & Solutions:**
    - **Wi-Fi Connection Failures**: Check credentials, router settings, ESP32 Wi-Fi capabilities
    - **Sensor Read Failures**: Check wiring, power supply, add delays between reads
    - **Web Server Not Responding**: Verify port 80 isn't blocked, check handleClient() in loop
    - **Multiple Client Issues**: ESP32 handles limited connections; may need connection management
    - **Memory Issues**: Large HTML strings can exhaust memory; optimize string handling


File Edit Selection View Go Run ···
Untitled (Workspace)
EXPLORER
UNTITLED (WORKSPACE)
assignment-aftermid
include
lib
src
main.cpp
test
.gitignore
diagram.json
platformio.ini
wokwi.toml
webserver1
.pio
.vscode
include
lib
src
main.cpp
test
.gitignore
diagram.json
platformio.ini
wokwi.toml
OUTLINE
TIMELINE
diagram.json webserver1
Wokwi Simulator
wokwi.toml
WOKWi Simulator
DHT22
ESP32
BTN
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Connecting to WiFi..
WiFi connected!
IP: 10.13.37.2
Default (webserver1)
Auto
Search



Question-2 Blynk Cloud Interfacing (blynk.cpp)

Part-A: Short Questions

1.Role of Blynk Template ID

The Template ID links the ESP32 code to a specific Blynk Cloud dashboard.
It must match the cloud template so data is sent to correct widgets and datastreams.

2. Difference between Template ID and Auth Token

Template ID identifies the dashboard design and configuration. Auth Token uniquely identifies and authorizes a specific ESP32 device.

3. Why DHT22 code fails with DHT11

DHT11 and DHT22 have different accuracy, timing, and data formats.
Using DHT22 code with DHT11 causes incorrect readings.
Key difference: DHT22 has higher accuracy and wider temperature range.

4. Virtual Pins in Blynk

Virtual Pins are software channels used to send data between ESP32 and Blynk Cloud.
They are preferred because they are flexible, hardware-independent, and cloud-friendly.

5. Why use BlynkTimer instead of delay()

BlynkTimer allows non-blocking execution, letting ESP32 handle Wi-Fi, cloud communication, and sensors simultaneously.
delay() blocks the program and causes connection issues.

Part-B: Long Question

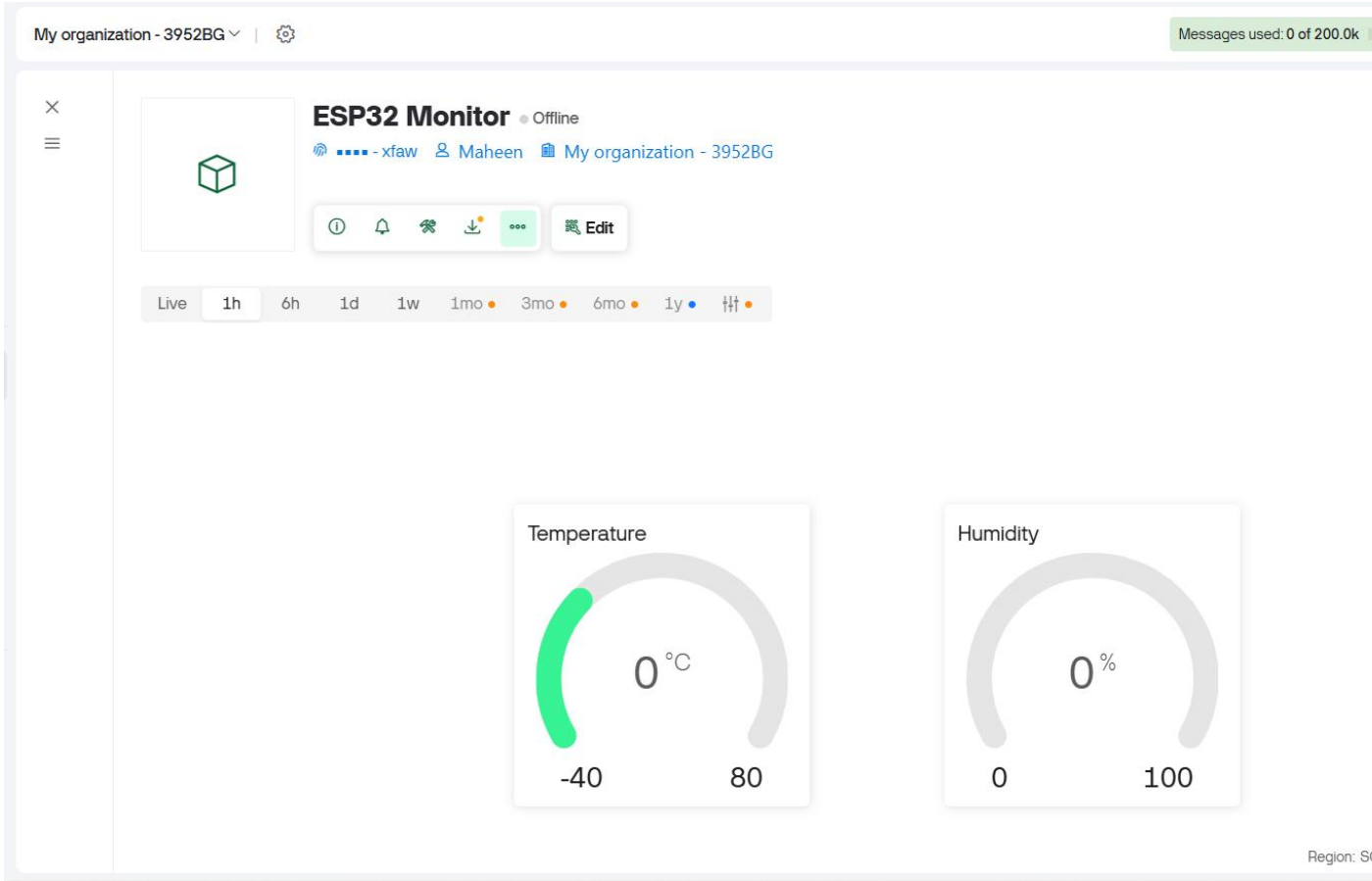
Explain the complete workflow of interfacing ESP32 with Blynk Cloud to display temperature and humidity values. Your answer should include: - Creation of Blynk Template and Datastreams - Role of Template ID, Template Name, and Auth Token - Sensor configuration issues (DHT11 vs DHT22) - Sending data using Blynk.virtualWrite() - Common problems faced during configuration and their solutions.

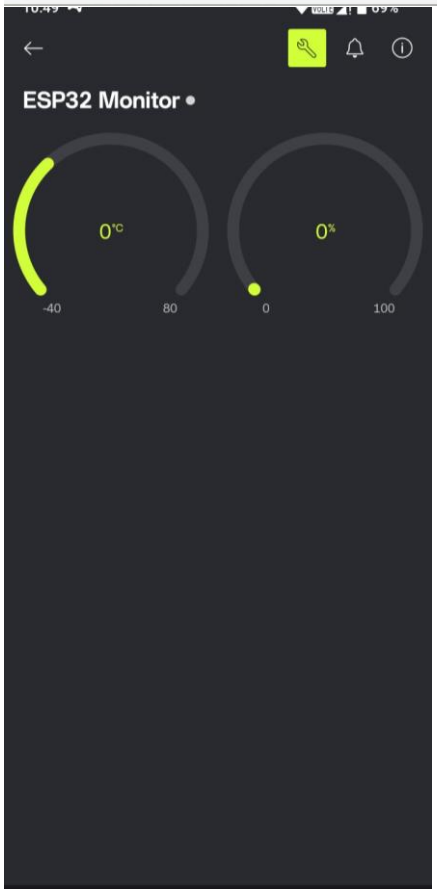
First, a Blynk Template is created with temperature and humidity datastreams. The Template ID, Template Name, and Auth Token are added to the ESP32 code.

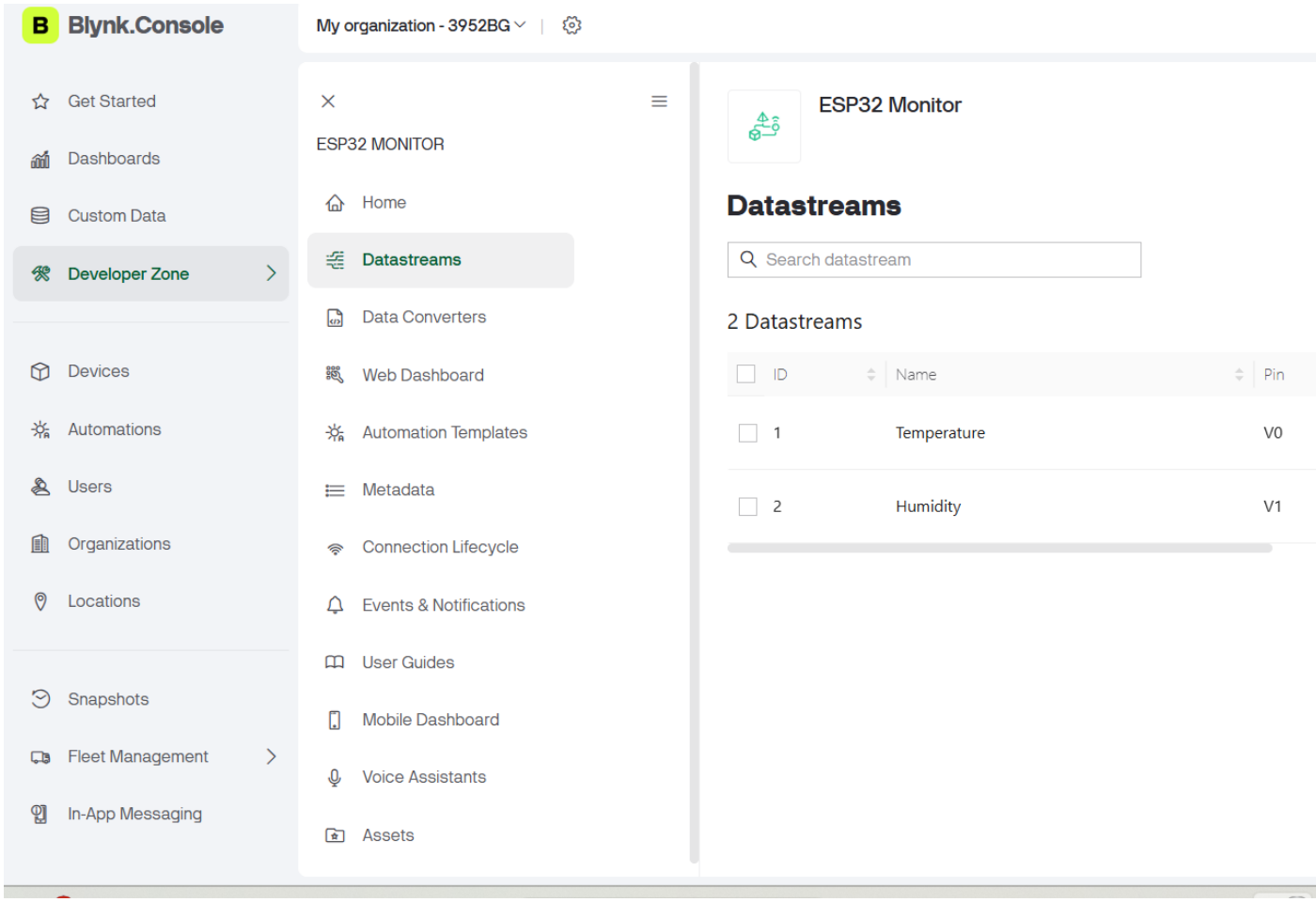
The ESP32 connects to Wi-Fi and then to the Blynk Cloud server. The DHT22 sensor is configured correctly to avoid sensor mismatch issues.

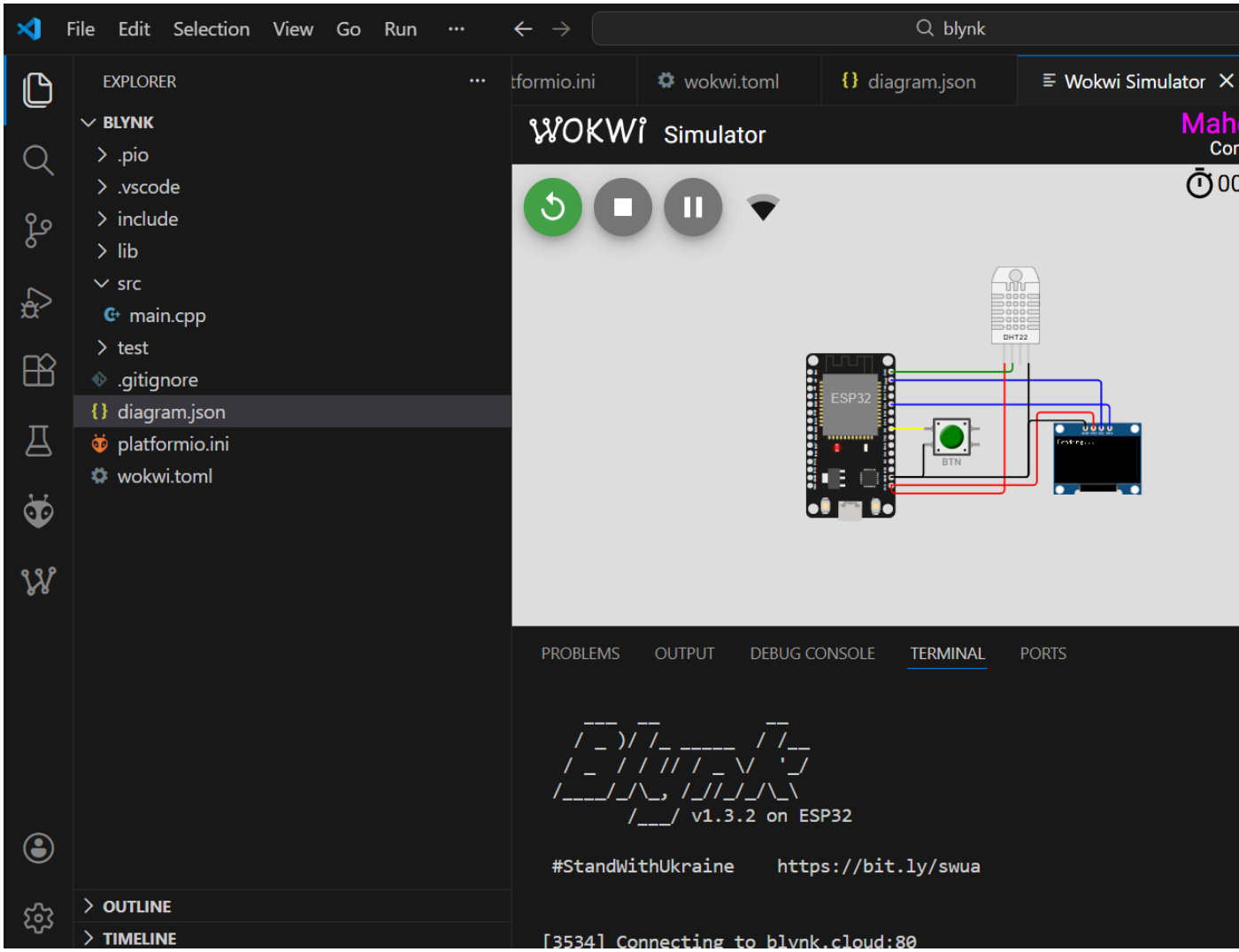
Sensor values are read and sent to Blynk using Blynk.virtualWrite() on virtual pins. These values are displayed on both the Blynk web dashboard and mobile application.

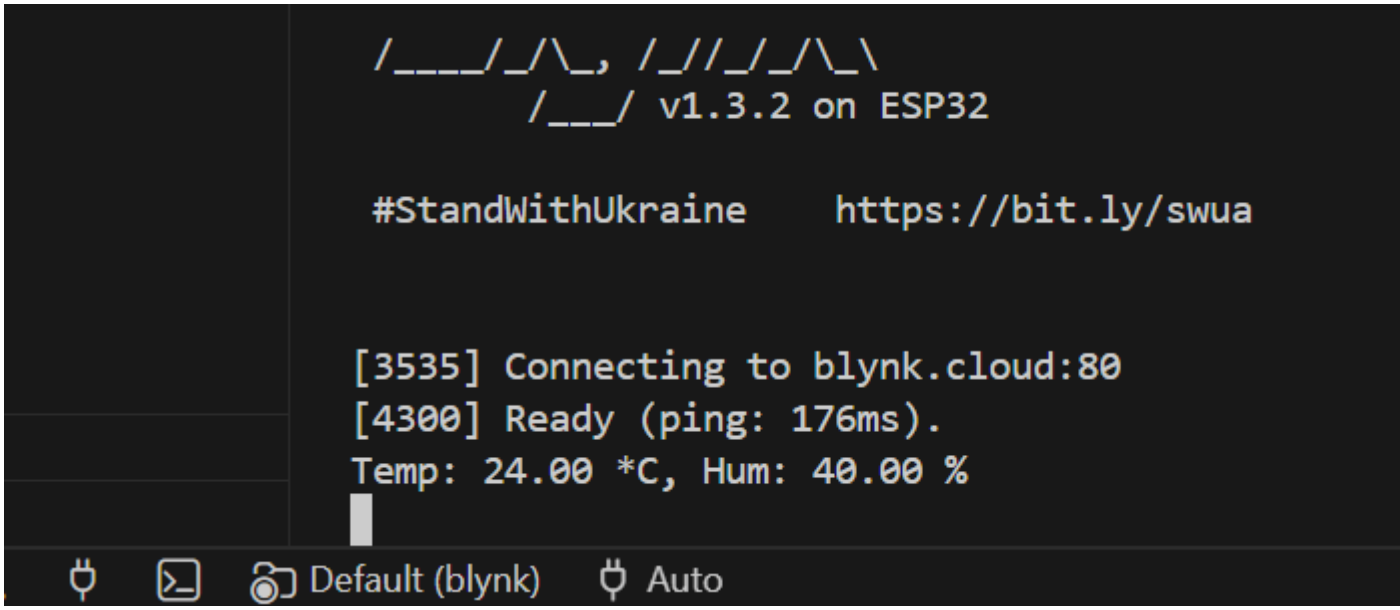
Common problems include incorrect virtual pin selection, wrong sensor type, and missing internet connection. These issues are solved by proper configuration and debugging.


My organization - 3952BG
Messages used: 0 of 200.0k
ESP32 Monitor
Offline
- xfaw
Maheen
My organization - 3952BG
Edit
Live
1h
6h
1d
1w
1mo
3mo
6mo
1y
Temperature
0 °C
-40
80
Humidity
0 %
0
100
Region:


ESP32 Monitor
0°C
0%
-40
80
0
100


Blynk.Console
My organization - 3952BG
Get Started
Dashboards
Custom Data
Developer Zone
Devices
Automations
Users
Organizations
Locations
Snapshots
Fleet Management
In-App Messaging
ESP32 MONITOR
Home
Datastreams
Data Converters
Web Dashboard
Automation Templates
Metadata
Connection Lifecycle
Events & Notifications
User Guides
Mobile Dashboard
Voice Assistants
Assets
ESP32 Monitor
Datastreams
Search datastream
2 Datastreams
ID
Name
Pin
1
Temperature
V0
2
Humidity
V1


File Edit Selection View Go Run
blynk
EXPLORER
BLYNK
.pio
.vscode
include
lib
src
main.cpp
test
.gitignore
diagram.json
platformio.ini
wokwi.toml
OUTLINE
TIMELINE
platformio.ini
wokwi.toml
diagram.json
Wokwi Simulator
WOKWi Simulator
ESP32
DHT22
BTN
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
/_____/_/_, /_//_/_/_\
/___/ v1.3.2 on ESP32
#StandWithUkraine https://bit.ly/swua
[3534] Connecting to blynk.cloud:80


/_____/_/_, /_//_/_/_\
/___/ v1.3.2 on ESP32
#StandWithUkraine https://bit.ly/swua
[3535] Connecting to blynk.cloud:80
[4300] Ready (ping: 176ms).
Temp: 24.00 *C, Hum: 40.00 %
Default (blynk)
Auto